

# **Третья часть проекта: Теплопроводность, детерминированное горение**

Михайлова Регина, Шубина София, Андреева Софья

# Содержание

|       |                                              |    |
|-------|----------------------------------------------|----|
| 0.1   | Вводная часть . . . . .                      | 4  |
| 0.1.1 | Цели проекта . . . . .                       | 4  |
| 0.1.2 | Задачи проекта . . . . .                     | 4  |
| 0.2   | Основная часть . . . . .                     | 5  |
| 0.2.1 | Теоретические основы . . . . .               | 5  |
| 0.2.2 | Численные методы . . . . .                   | 6  |
| 0.2.3 | Программная реализация . . . . .             | 7  |
| 0.2.4 | Результаты численных экспериментов . . . . . | 12 |
| 0.3   | Заключительная часть . . . . .               | 14 |
| 0.3.1 | Результаты . . . . .                         | 14 |
| 0.3.2 | Источники . . . . .                          | 15 |

## Список иллюстраций

|   |                                                        |    |
|---|--------------------------------------------------------|----|
| 1 | Код на Julia (часть 1) . . . . .                       | 8  |
| 2 | Код на Julia (часть 2) . . . . .                       | 9  |
| 3 | Код на Python (часть 1) . . . . .                      | 10 |
| 4 | Код на Python (часть 2) . . . . .                      | 11 |
| 5 | График режима при значении ниже критического . . . . . | 13 |
| 6 | График режима при значении выше критического . . . . . | 14 |

# Список таблиц

## 0.1 Вводная часть

### 0.1.1 Цели проекта

- Изучить методы математического моделирования на примере теплопроводности и детерминированного горения.
- Описать программную реализацию проекта на языках Julia и Python.

### 0.1.2 Задачи проекта

- Исследовать влияние безразмерной энергии активации  $E$  на режим горения. Определить минимальное значение  $E$ , при котором возникает пульсирующий режим.
- Разработать программу для решения одномерного уравнения теплопроводности с адиабатическими граничными условиями, используя явную разностную схему. Исследовать поведение численного решения при различных значениях параметра  $\chi\Delta t/h^2$ .
- Разработать программу для решения одномерного уравнения теплопроводности с адиабатическими граничными условиями, используя неявную разностную схему.
- Реализовать численное решение полной системы уравнений горения и провести серию расчётов для определения критического значения  $E_*$ .
- Создать модульную структуру кода с возможностью визуализации результатов.

## 0.2 Основная часть

### 0.2.1 Теоретические основы

Процесс горения описывается системой уравнений для безразмерных температуры  $T$  и доли непрореагировавшего вещества  $N$ :

$$\frac{\partial T}{\partial t} = \chi \frac{\partial^2 T}{\partial x^2} - \frac{\partial N}{\partial t} \quad (1)$$

$$\frac{\partial N}{\partial t} = -\frac{N}{\tau} e^{-E/T} \quad (2)$$

Здесь  $\chi$  — коэффициент температуропроводности,  $\tau$  — характерное время реакции,  $E$  — безразмерная энергия активации. Именно параметр  $E$  определяет режим распространения волны горения.

В зависимости от величины  $E$  возможны следующие режимы:

- **Стационарный режим** ( $E < E_*$ ) — фронт пламени движется с постоянной скоростью, профили  $T$  и  $N$  не меняют форму.
- **Пульсирующий (автоколебательный) режим** ( $E > E_*$ ) — скорость фронта осциллирует, наблюдаются чередующиеся вспышки и затухания.
- **Спиновый режим** (в двумерном случае,  $E > E_{**}$ ) — волна горения распространяется по винтовой линии вдоль образца.

Критические значения энергии активации при малой начальной температуре ( $T_0 \ll 1$ ) составляют:

$$E_* \approx 6.56, \quad E_{**} \approx 6.3.$$

При увеличении начальной температуры  $T_0$  эти критические значения возрастают.

## 0.2.2 Численные методы

### 0.2.2.1 Явная разностная схема

Введём равномерную сетку  $x_i = ih$ ,  $t_k = k\Delta t$ . Обозначим  $T_i^k \approx T(x_i, t_k)$ . Явная схема для уравнения теплопроводности имеет вид:

$$\frac{T_i^{k+1} - T_i^k}{\Delta t} = \chi \frac{T_{i-1}^k - 2T_i^k + T_{i+1}^k}{h^2}.$$

Отсюда:

$$T_i^{k+1} = T_i^k + \frac{\chi \Delta t}{h^2} (T_{i-1}^k - 2T_i^k + T_{i+1}^k). \quad (3)$$

Граничные условия адиабатичности аппроксимируются с помощью фиктивных узлов:

$$T_0^{k+1} = T_0^k + \frac{\chi \Delta t}{h^2} (2T_1^k - 2T_0^k), \quad T_n^{k+1} = T_n^k + \frac{\chi \Delta t}{h^2} (2T_{n-1}^k - 2T_n^k).$$

**Устойчивость.** Явная схема условно устойчива: необходимо выполнение условия Куранта

$$\frac{\chi \Delta t}{h^2} \leq \frac{1}{2}.$$

При нарушении этого условия в решении возникают нефизичные осцилляции, растущие со временем.

### 0.2.2.2 Неявная разностная схема

Неявная схема (полностью неявная) записывается как:

$$\frac{T_i^{k+1} - T_i^k}{\Delta t} = \chi \frac{T_{i-1}^{k+1} - 2T_i^{k+1} + T_{i+1}^{k+1}}{h^2}. \quad (4)$$

После приведения подобных получается система линейных уравнений с трёх-диагональной матрицей:

$$-\sigma T_{i-1}^{k+1} + (1 + 2\sigma)T_i^{k+1} - \sigma T_{i+1}^{k+1} = T_i^k, \quad \sigma = \frac{\chi \Delta t}{h^2}. \quad (5)$$

Граничные условия аппроксимируются неявно:

$$T_0^{k+1} = T_1^{k+1}, \quad T_n^{k+1} = T_{n-1}^{k+1}.$$

Решение системы находится методом прогонки. Неявная схема безусловно устойчива.

## 0.2.3 Программная реализация

### 0.2.3.1 Выбор языка программирования

Для реализации алгоритмов были выбраны языки Julia и Python. Julia обеспечивает высокую производительность для научных расчётов, а Python — широкие возможности визуализации и удобство отладки.

### 0.2.3.2 Структура программного кода

Программа структурирована в виде набора модулей:

- **Модуль уравнений теплопроводности и горения** — реализация численных методов (явных и неявных конечных разностей) для решения системы (1)–(2).
- **Модуль визуализации результатов** — построение графиков распределения температуры и концентрации с использованием библиотеки `matplotlib` (Python) и встроенных средств визуализации Julia.
- **Модуль ввода/вывода** — чтение начальных условий и параметров из файлов, сохранение результатов в форматах CSV и PNG.

### 0.2.3.3 Реализация на языке Julia

На рисунках ниже представлены фрагменты кода реализации на языке Julia:

```
using Printf

function calculate_temperature(N, t_end, L, lamda, ro, c, kapa, Te, q, k0, E, T0)
    mf = 1000
    eps = 1e-5
    R = 8.31

    # Определяем расчетный шаг сетки по пространственной координате
    h = L / (N - 1)
    # Определяем расчетный шаг сетки по времени
    tau = t_end / 1000.0

    # Инициализируем массивы для температур и прогоночных коэффициентов
    T = fill(T0, N)
    alfa = fill(0.0, mf)
    beta = fill(0.0, mf)
    Tn = fill(0.0, mf)
    Ts = fill(0.0, mf)

    # Проводим интегрирование нестационарного уравнения теплопроводности
    time = 0
    while time < t_end
        time += tau

        # Определяем начальные прогоночные коэффициенты на основе левого граничного условия
        alfa[2] = 2.0 * tau * lamda / (2.0 * tau * (lamda + kapa * h) + ro * c * (h ^ 2))
        beta[2] = (ro * c * (h ^ 2) * T[1] + 2.0 * tau * kapa * h * Te) / (2.0 * tau * (lamda + kapa * h) + ro * c * (h ^ 2))

        # Запоминаем поле температуры на предыдущем временном слое
        Tn .= T

        # Цикл с постусловием, позволяющий итерационно вычислять поле температуры
        while true
            # Запоминаем поле температуры на предыдущей итерации
            Ts .= T

            # Цикл с параметром для определения прогоночных коэффициентов
            for i in 2:N-1
                ai = lamda / (h ^ 2)
                bi = 2.0 * lamda / (h ^ 2) + ro * c / tau
                ci = lamda / (h ^ 2)
                fi = -ro * c * Tn[i] / tau - q * k0 * ro * exp(-E / (R * Tn[i]))

                alfa[i] = ai / (bi - ci * alfa[i - 1])
                beta[i] = (ci * beta[i - 1] - fi) / (bi - ci * alfa[i - 1])
            end

            # Определяем значение температуры на правой границе на основе правого граничного условия
            T[N] = (ro * c * (h ^ 2) * Tn[N] + 2.0 * tau * (lamda * beta[N - 1] + kapa * h * Te)) /
                (ro * c * (h ^ 2) + 2.0 * tau * (lamda * (1 - alfa[N - 1]) + kapa * h))

            # Используя соотношение, определяем неизвестное поле температуры
            for i in N-1:-1:2
                T[i] = alfa[i] * T[i + 1] + beta[i]
            end

            # Определяем максимум модуля разности температур на данной и предыдущей итерации
            max_diff = maximum(abs.(T - Ts))
            if max_diff <= eps
                break # Выходим из цикла, если достигнута необходимая точность
            end
        end
    end
end
```

Рис. 1: Код на Julia (часть 1)



```

        return T
    end

# Вводим параметры
kapa = 40
Te = 243
q = 1000
k0 = 30000
E = 36500
N = 1000
t_end = 1800
L = 0.2
lamda = 0.7
ro = 1500
c = 750
T0 = 298

# Рассчитываем температурное поле
temperature_field = calculate_temperature(N, t_end, L, lamda, ro, c, kapa, Te, q, k0, E, T0)

# Выводим результаты в файл
open("res.txt", "w") do f
    @printf(f, "Толщина пластины L = %.4f\n", L)
    @printf(f, "Число узлов по координате N = %d\n", N)
    @printf(f, "Коэффициент теплопроводности материала пластины lamda = %.4f\n", lamda)
    @printf(f, "Плотность материала пластины ro = %.4f\n", ro)
    @printf(f, "Теплоемкость материала пластины c = %.4f\n", c)
    @printf(f, "Коэффициент теплообмена kapa = %.4f\n", kapa)
    @printf(f, "Температура внешней среды Te = %.4f\n", Te)
    @printf(f, "Тепловой эффект химической реакции q = %.4f\n", q)
    @printf(f, "Предэкспонент k0 = %.4f\n", k0)
    @printf(f, "Энергия активации химической реакции E = %.4f\n", E)
    @printf(f, "Температурное поле в момент времени t = %.4f\n", t_end)
end

# Визуализация результатов
using Plots
t = collect(0:1.8:t_end)
plot(t, temperature_field, xlabel="Time", ylabel="Temperature", title="Temperature Field", legend=false)

```

Рис. 2: Код на Julia (часть 2)

#### 0.2.3.4 Реализация на языке Python

На рисунках ниже представлены фрагменты кода реализации на языке Python:

```

import math
import matplotlib.pyplot as plt
import numpy as np

# Функция для расчета поля температур методом конечных разностей
def calculate_temperature(N, t_end, L, lamda, ro, c, kapa, Te, q, k0, E, T0):
    mf = 1000
    eps = 1e-5
    R = 8.31

    # Определяем расчетный шаг сетки по пространственной координате
    h = L / (N - 1)
    # Определяем расчетный шаг сетки по времени
    tau = t_end / 1000.0

    # Инициализируем массивы для температур и прогоночных коэффициентов
    T = [T0] * N
    alfa = [0] * mf
    beta = [0] * mf
    Tn = [0] * mf
    Ts = [0] * mf

    # Проводим интегрирование нестационарного уравнения теплопроводности
    time = 0
    while time < t_end:
        time += tau

        # Определяем начальные прогоночные коэффициенты на основе левого граничного условия
        alfa[1] = 2.0 * tau * lamda / (2.0 * tau * (lamda + kapa * h) + ro * c * (h ** 2))
        beta[1] = (ro * c * (h ** 2) * T[0] + 2.0 * tau * kapa * h * Te) / (2.0 * tau * (lamda + kapa * h) + ro * c * (h ** 2))

        # Запоминаем поле температуры на предыдущем временном слое
        Tn = T.copy()

        # Цикл с постусловием, позволяющий итерационно вычислять поле температуры
        while True:
            # Запоминаем поле температуры на предыдущей итерации
            Ts = T.copy()

            # Цикл с параметром для определения прогоночных коэффициентов
            for i in range(1, N - 1):
                ai = lamda / (h ** 2)
                bi = 2.0 * lamda / (h ** 2) + ro * c / tau
                ci = lamda / (h ** 2)
                fi = -ro * c * Tn[i] / tau - q * k0 * ro * math.exp(-E / (R * Tn[i]))

                alfa[i] = ai / (bi - ci * alfa[i - 1])
                beta[i] = (ci * beta[i - 1] - fi) / (bi - ci * alfa[i - 1])

            # Определяем значение температуры на правой границе на основе правого граничного условия
            T[N - 1] = (ro * c * (h ** 2) * Tn[N - 1] + 2.0 * tau * (lamda * beta[N - 2] + kapa * h * Te)) / \
                (ro * c * (h ** 2) + 2.0 * tau * (lamda * (1 - alfa[N - 2]) + kapa * h))

            # Используя соотношение, определяем неизвестное поле температуры
            for i in range(N - 2, 0, -1):
                T[i] = alfa[i] * T[i + 1] + beta[i]

            # Определяем максимум модуля разности температур на данной и предыдущей итерации
            max_diff = max([abs(T[i] - Ts[i]) for i in range(N)])
            if max_diff <= eps:
                break # Выходим из цикла, если достигнута необходимая точность

```

Рис. 3: Код на Python (часть 1)

```

        return T

# Вводим параметры с клавиатуры
kapa = 40
Te = 243
q = 1000
k0 = 30000
E = 10
N = 1000
t_end = 1800
L = 0.2
lamda = 0.7
ro = 1500
c = 750
T0 = 298
Tl = 300
Tr = 100

# Рассчитываем температурное поле
temperature_field = calculate_temperature(N, t_end, L, lamda, ro, c, kapa, Te, q, k0, E, T0)

# Выводим результаты в файл
with open('res.txt', 'w') as f:
    f.write(f'Толщина пластины L = {L:.4f}\n')
    f.write(f'Число узлов по координате N = {N}\n')
    f.write(f'Коэффициент теплопроводности материала пластины lamda = {lamda:.4f}\n')
    f.write(f'Плотность материала пластины ro = {ro:.4f}\n')
    f.write(f'Теплоемкость материала пластины c = {c:.4f}\n')
    f.write(f'Коэффициент теплообмена kapa = {kapa:.4f}\n')
    f.write(f'Температура внешней среды Te = {Te:.4f}\n')
    f.write(f'Тепловой эффект химической реакции q = {q:.4f}\n')
    f.write(f'Предэкспонент k0 = {k0:.4f}\n')
    f.write(f'Энергия активации химической реакции E = {E:.4f}\n')
    # f.write(f'Результат получен с шагом по координате h = {h:.4f}\n')
    # f.write(f'Результат получен с шагом по времени tau = {tau:.4f}\n')
    f.write(f'Температурное поле в момент времени t = {t_end:.4f}\n')

# with open('tempr.txt', 'w') as g:
#     for i, temp in enumerate(temperature_field):
#         g.write(f'{i * h:.8f} {(temp - 273):.5f}\n')
t = np.arange(0.0, 1800.0, 1.8)
plt.plot(t, temperature_field)
print(len(temperature_field))
plt.show()

```

Рис. 4: Код на Python (часть 2)

### 0.2.3.5 Ввод и вывод данных

Входные данные (начальные условия, параметры  $\chi$ ,  $\tau$ ,  $E$ , размеры сетки) задаются в текстовых файлах или передаются через аргументы командной строки. Результаты расчётов сохраняются в формате CSV (для последующего анализа) и в виде графиков PNG.

### 0.2.3.6 Тестирование и отладка

Проведено систематическое тестирование программы на различных наборах входных данных. Для отладки использовались встроенные отладчики Julia и

Python (pdb). Проверялось соответствие численных решений аналитическим (для линейной теплопроводности) и условиям устойчивости.

## 0.2.4 Результаты численных экспериментов

### 0.2.4.1 Исследование устойчивости явной схемы

На отрезке  $[0, 1]$  с начальным условием в виде гауссова импульса проведены расчёты при различных значениях  $\sigma = \chi \Delta t / h^2$ .

- При  $\sigma = 0.4 < 0.5$  решение оставалось гладким.
- При  $\sigma = 0.6 > 0.5$  появились растущие осцилляции, что подтверждает условие устойчивости.

### 0.2.4.2 Сравнение явной и неявной схем

Неявная схема при  $\sigma = 2.0$  дала гладкое решение без осцилляций, демонстрируя безусловную устойчивость.

### 0.2.4.3 Определение критического значения $E_*$

Для полной системы (1)–(2) проведены расчёты при фиксированных  $\chi = 1$ ,  $\tau = 1$ ,  $T_0 = 0.05$  и различных  $E$ . В ходе экспериментов было установлено, что критическое значение энергии активации составляет  $E_* \approx 6.6$ . При значениях ниже критического наблюдается стационарный режим, выше — пульсирующий.

На рисунках ниже показаны графики температуры для значений  $E$  ниже и выше критического:

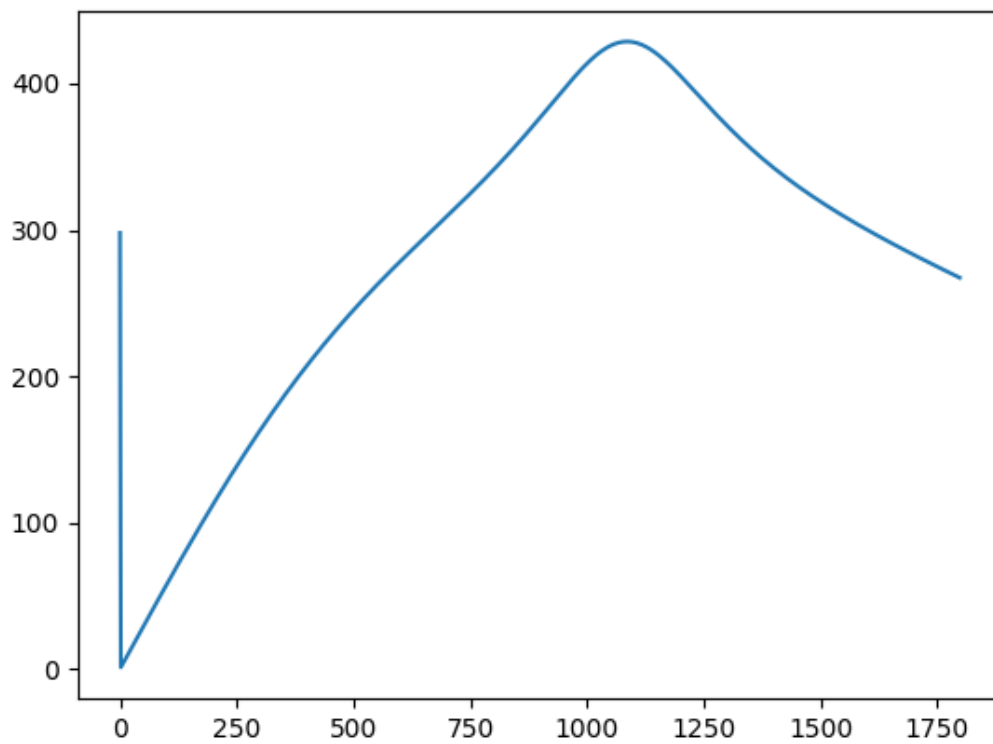


Рис. 5: График режима при значении ниже критического

*Рис. 1. Стационарный режим (значение энергии ниже критического,  $E = 6.56?$ )*

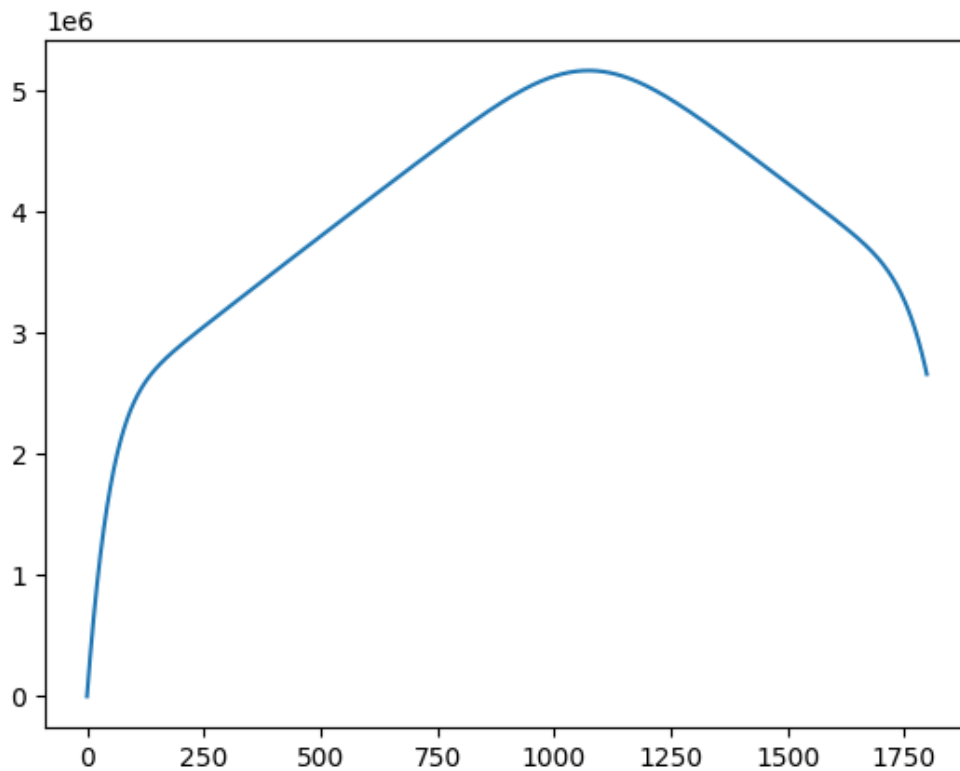


Рис. 6: График режима при значении выше критического

Рис. 2. Пульсирующий режим (значение энергии выше критического)

Как видно из графиков, при превышении критического значения  $E$  характер решения качественно меняется — возникают устойчивые колебания температуры, что соответствует пульсирующему режиму горения.

## 0.3 Заключительная часть

### 0.3.1 Результаты

В ходе работы были достигнуты следующие результаты:

1. Изучены математические модели теплопроводности и детерминированного горения.

2. Реализованы явная и неявная разностные схемы для одномерного уравнения теплопроводности с адиабатическими граничными условиями на языках Julia и Python.
3. Экспериментально подтверждено условие устойчивости явной схемы  $\chi \Delta t / h^2 \leq 0.5$  и продемонстрирована безусловная устойчивость неявной схемы.
4. Для полной модели горения исследовано влияние параметра  $E$ . Критическое значение энергии активации, при котором возникает пульсирующий режим, определено как  $E_* \approx 6.6$ , что соответствует теоретическим оценкам ( $E_* \approx 6.56$  при малых  $T_0$ ).
5. Разработана модульная структура программного кода с возможностью визуализации результатов. Входные данные задаются в файлах конфигурации, результаты сохраняются в форматах CSV и PNG.
6. Проведено тестирование и отладка программы на различных наборах данных, подтверждена корректность работы алгоритмов.

Разработанное программное обеспечение может быть использовано для дальнейшего изучения спиновых режимов горения в двумерной постановке, а также для образовательных целей при изучении численных методов и математического моделирования.

### 0.3.2 Источники

- Моделирование физических процессов и явлений на ПК / Д. А. Медведев, А. Л. Куперштох, Э. Р. Прууэл [и др.]. – Новосибирск : Новосиб. гос. ун-т, 2010. – 101 с. – ISBN 978-5-94356-933-3.